

MORDERN APPLICATION DEVELOPMENT-1

APP DEVELOPMENT PROJECT REPORT

By
NAME: MRIDUL GOYAL
Roll No.: 24F2003142
Email.:24f2003142@ds.study.iitm.ac.in



Indian Institute Of Technology, Madras

Chennai-711103, Tamil Nadu, India

Date: 06/03/2026

About Me

I am an undergraduate engineering student with a strong interest in software development and real-world application building. I enjoy designing full-stack systems, especially those involving database management, user interfaces, and workflow automation. This project allowed me to strengthen my understanding of Flask, SQLAlchemy, and practical hospital workflow modeling through hands-on development.

1. Project Details

Project Title: Placyx – Placement Management Portal

Problem Statement:

Campus placement processes typically involve multiple stakeholders such as students, companies, and placement administrators. In many institutions, these activities are managed using spreadsheets, emails, or fragmented tools, which can lead to inefficiencies and lack of transparency.

Common issues in traditional placement management include:

1. Difficulty in tracking student applications across multiple companies
2. Lack of centralized visibility of placement drives
3. Inefficient approval workflows for job postings
4. Limited monitoring of placement statistics and analytics

Manual coordination between students, companies, and administrators

To address these issues, a centralized web application is required that can manage the placement lifecycle digitally while ensuring secure access and organized workflows.

Approach:

The Placyx Placement Portal was developed as a Flask-based web application implementing a structured placement management workflow. The system supports three primary user roles: Administrator, Company, and Student.

The backend logic is implemented using Flask with SQLite and SQLAlchemy for database management. Each entity in the system—such as users, students, companies, job posts, and applications—is represented as a relational database table.

The frontend uses Jinja2 templates combined with Bootstrap to provide a responsive and interactive user interface. Dynamic pages fetch and display data directly from the database, enabling real-time updates across dashboards.

The system implements:

1. Role-based authentication for Admin, Company, and Student users
2. Company registration and placement drive posting
3. Admin approval workflow for job postings
4. Student job browsing and application system

5. Company candidate evaluation and selection workflow
6. Placement tracking and analytics dashboards
7. Automatic closure of placement drives based on deadlines

The overall approach focuses on building a modular and maintainable system that models real placement workflows while ensuring data consistency and secure user access.

2. AI/LLM Declaration

During the development of this project, I used ChatGPT (GPT-5) as a supporting tool in the following ways:

- Improving code readability and restructuring certain route functions
- Writing boilerplate SQLAlchemy model snippets
- Generating documentation-friendly descriptions for APIs and workflows
- Helping with HTML/Jinja template formatting suggestions
- Clarifying error messages and debugging approaches

The total extent of AI usage is approximately **10–15%**, limited to guidance, explanation, and improving code style.

All major components — including system design, database structure, route logic, CRUD operations, and integration — were implemented manually by me.

No AI-generated code was directly copied without validation or modification.

I confirm that the entire project logic, testing, debugging, and final implementation have been completed independently.

3. Technologies and Frameworks Used

Technology / Library	Purpose / Role in Project
Flask	Core backend framework used for routing, request handling, and application logic.
SQLite	Lightweight relational database used to store users, doctors, patients, appointments, departments, and visit records
SQLAlchemy	ORM used to map Python classes to database tables and simplify CRUD operations.
Jinja2	Template engine for rendering dynamic HTML pages based on user roles and backend data.
Bootstrap 5	Used for responsive UI design, layout structure, modals, tables, and easy styling.
HTML5 & CSS3	Base technologies for frontend page structure and custom styling
SQLite Foreign Keys	Ensures relational integrity between patients, doctors, appointments, and visits

Flask-Login	User authentication and session management
Werkzeug	Password hashing and file upload handling
Chart.js	Used for analytics dashboards and data visualization
JavaScript (vanilla)	Used for UI interactions and confirmation dialogs

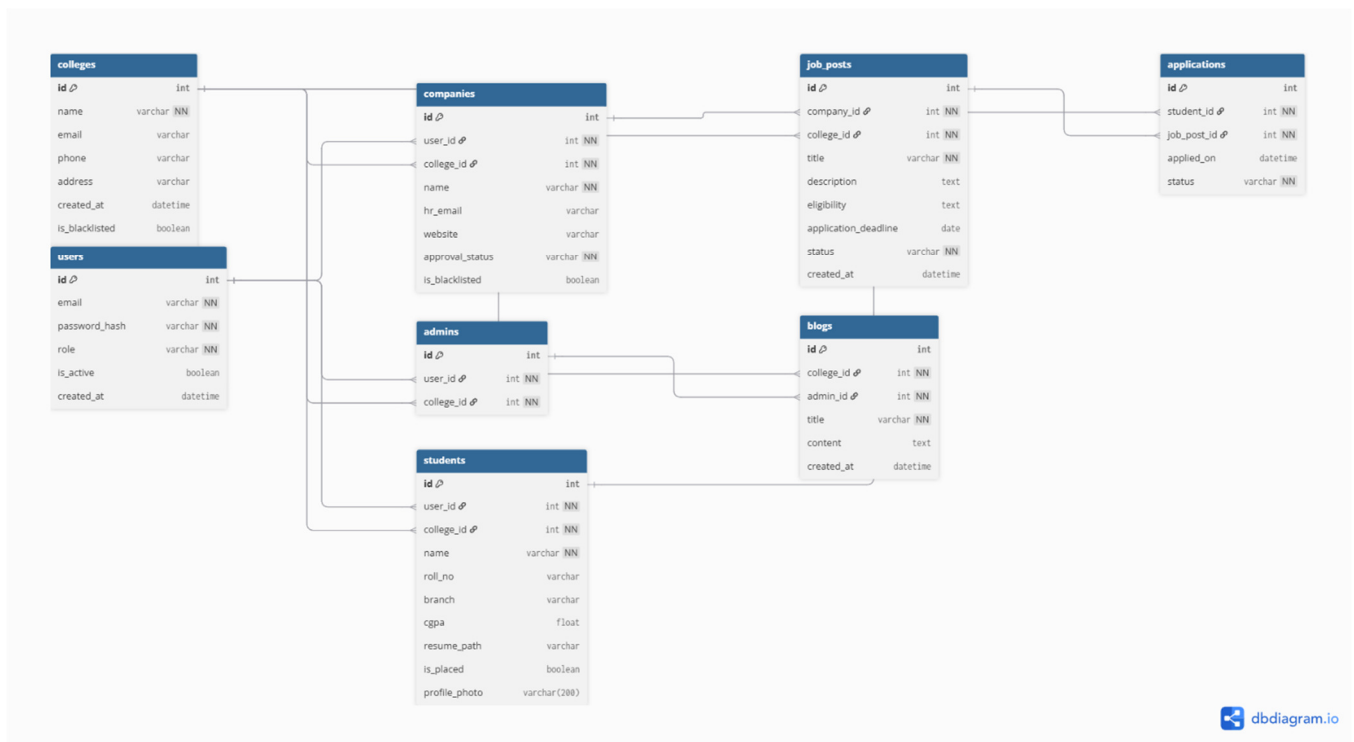
4. Database Schema / ER Diagram

Tables:

1. **users:** This table stores the login credentials and basic identity information of every system user.
2. **admins:** This Table contains the 3 fields user id and college id. Bridging gap between college and admins.
3. **colleges:** It contains personal address and other details of the registering colleges but this is not being currently used as only one admin is permitted this database schema is given to enable new colleges to register and use as admins individually.
4. **students:** Contains personal and academic details of students participating in placement drives.
5. **companies:** Stores company profile information including name, website, and company logo.
6. **job_posts:** Represents placement drives created by companies.
7. **applications:** Tracks job applications submitted by students and their current selection status.
8. **Blogs:** this table represents if any posts or blog the college wants to post for students can be stored here (not currently used as this is out of scope of current wireframe used for future development of project)

Relationships:

- One-to-One → **users** → **students**
- One-to-One → **users** → **companies**
- One-to-Many → **companies** → **job_posts**
- One-to-Many → **students** → **applications**
- One-to-Many → **job_posts** → **applications**



5. API Resource Endpoints

Endpoint	Method(s)	Purpose / Key Inputs
/	GET	Public landing page displaying the platform introduction (renders index.html).
/login	GET, POST	User authentication. POST validates email and password and redirects to role-based dashboard.
/logout	GET	Logs out the current user and redirects to login page.
/register/student	GET, POST	Student registration form. POST creates user account and student profile.
/register/company	GET, POST	Company registration form. POST creates company account pending admin approval.
/admin/dashboard	GET	Admin analytics dashboard displaying statistics for students, companies, jobs, and placements.
/admin/placements	GET	Displays list of all placed students.
/admin/companies	GET	Shows companies waiting for approval.
/admin/company/approve/<company_id>	GET	Admin approves a company registration.
/admin/company/reject/<company_id>	GET	Admin rejects a company registration.
/admin/manage-companies	GET	Displays all companies with search functionality.
/admin/company/deactivate/<company_id>	GET	Deactivates a company account.

/admin/company/activate/<company_id>	GET	Reactivates a company account.
/admin/students	GET	Displays student list with search functionality.
/admin/student/deactivate/<student_id>	GET	Deactivates a student account.
/admin/student/activate/<student_id>	GET	Reactivates a student account.
/admin/students/search	GET	Search students by name, roll number, branch, or CGPA.
/admin/companies/search	GET	Redirect search request for companies.
/admin/job-posts	GET	Displays job posts awaiting admin approval.
/admin/job/approve/<job_id>	GET	Approves a job post submitted by a company.
/admin/job/reject/<job_id>	GET	Rejects a job post submitted by a company.
/admin/jobs	GET	Displays all placement drives posted in the system.
/admin/applications	GET	Displays all job applications across companies.
/company/dashboard	GET	Company dashboard showing analytics for jobs and applications.
/company/job/create	GET, POST	Company creates a placement drive.
/company/jobs	GET	Displays all job posts created by the company.
/company/job/<job_id>/applications	GET	View applicants for a specific job post.
/company/application/<app_id>/shortlist	GET	Shortlists a candidate application.
/company/application/<app_id>/reject	GET	Rejects a candidate application.
/company/application/<app_id>/select	GET	Selects a candidate and closes the placement drive.
/company/application/<app_id>/<status>	GET	Updates application status (SHORTLISTED, REJECTED, SELECTED).
/company/application/<app_id>	GET	Displays detailed profile of a candidate.
/company/job/<job_id>/close	GET	Company manually closes a placement drive.
/company/profile	GET, POST	Update company profile details such as website, HR email, and logo.
/student/dashboard	GET	Displays all approved placement drives available for the student's college.
/student/job/<job_id>	GET	View detailed job description and company details.
/student/job/apply/<job_id>	GET	Student applies for a job post.
/student/applications	GET	Displays all job applications submitted by the student.
/student/profile	GET, POST	Update student profile, upload photo, and resume.

6. Architecture and Features

Architecture Overview

- **app.py** – Main Flask application entry point containing route definitions, authentication logic, placement workflow handling, and application initialization.
- **models.py** – Contains SQLAlchemy database models defining entities such as Users, Students, Companies, JobPosts, Applications, Blogs, and Colleges along with their relationships.
- **config.py** – Configuration file used to define application settings such as database connection, secret keys, and environment configuration.
- **/templates** – Jinja2 HTML templates used to render dynamic pages for Admin, Company, and Student dashboards and other application views.
- **/static/uploads** – Stores uploaded files such as student profile photos, resumes, and company logos.
- **requirements.txt** – Contains all Python dependencies required to run the application.
- **local.ps1** – Script used to initialize the environment, install dependencies, create the database if missing, and launch the Flask application locally.
- **venv/** – Virtual environment directory containing isolated Python packages for the project.
- **Instance/** – This folder contains the database for the placyx application allo the details are stored in the database stored in this folder.

Implemented Features

1. **User Authentication System**
Secure login system with password hashing and session management using Flask-Login.
2. **Role-Based Access Control**
Three distinct user roles are implemented:
 - i. Administrator
 - ii. Company
 - iii. StudentEach role has separate dashboards and permissions.
3. **Student Module**
Students can:
 - i. Register accounts
 - ii. View approved placement drives
 - iii. View detailed job descriptions
 - iv. Apply for placement drives
 - v. Track application status
 - vi. Upload profile photo and resume
4. **Company Module**
Companies can:
 - i. Register company accounts
 - ii. Create placement drives
 - iii. View applicant lists
 - iv. View candidate profiles

- v. Shortlist or reject candidates
- vi. Select final candidates
- vii. Close placement drives
- viii. Manage company profile and logo

5. **Admin Module**

Administrators can:

- i. Approve or reject company registrations
- ii. Approve or reject job posts
- iii. View all placement drives
- iv. View all student applications
- v. Search students and companies
- vi. Deactivate or reactivate user accounts
- vii. Monitor placement statistics

6. **Placement Workflow Management**

The system manages the entire placement lifecycle:

Company Job Post → Admin Approval → Student Applications → Company Shortlisting → Candidate Selection → Placement Confirmation.

7. **Analytics Dashboard**

Analytics are displayed using Chart.js showing:

- i. Number of students
- ii. Number of companies
- iii. Job posting statistics
- iv. Placement results

8. **File Upload System**

Students and companies can upload files such as:

- i. Student profile photos
- ii. Student resumes
- iii. Company logos

9. **Automatic Placement Logic**

When a company selects a candidate:

- i. Student status is marked as **placed**
- ii. All other applicants for the same job are automatically rejected
- iii. The placement drive is automatically closed

7. Video Presentation:

Drive Link:

<https://drive.google.com/file/d/1De48jKnT0uuCHuKiaqZ6zGnWWmgK7esl/view?usp=sharing>

(Accessible to all with "View" permission.)